Soccer: Encryption with Generated Expanders

Abstract Soccer is a password-based symmetric encryption construction, originally developed for a new archiving tool motivated by recoverability. This paper examines the encryption itself: material generation, plan construction, and reversible data transformation. Setup prepares a reproducible key-and-control plan for 1 MiB inputs. Subsequent encryption or decryption calls apply that plan without carrying evolving setup state between inputs. Setup expands password- and nonce-derived input into a stack of pseudorandom material intended for cryptographic use. Its construction combines ARX-based loops, M88 matrix transformations, and transient KDF state. Intermediate results derive the salts and control constants used by subsequent KDF stages across six functional domains. After expansion, folding draws on the full material stack to determine key-material allocation, masks, rotations, and cipher variants. These choices populate a constrained schedule of reversible symmetric transformations at whole-input, half-input, and quarter-input scales. Soccer emphasizes the construction and coordination of these layers. Mutation and movement operations are interleaved with rotations, with the aim of retaining repeated mixing even when rotation offsets are complementary. Mean-Machine generates expander implementations with explicit data-flow constraints; its loop-development process examines influence propagation, coverage, and structural robustness. Implementations specialized for the input geometry connect expanded material to the configured cipher plan. The proposed contribution is this composition of generated expanders, interacting material lanes, and constrained reversible operations, developed through implementation and iterative experiments. The security of the expanders and the resulting composition remains subject to independent analysis.

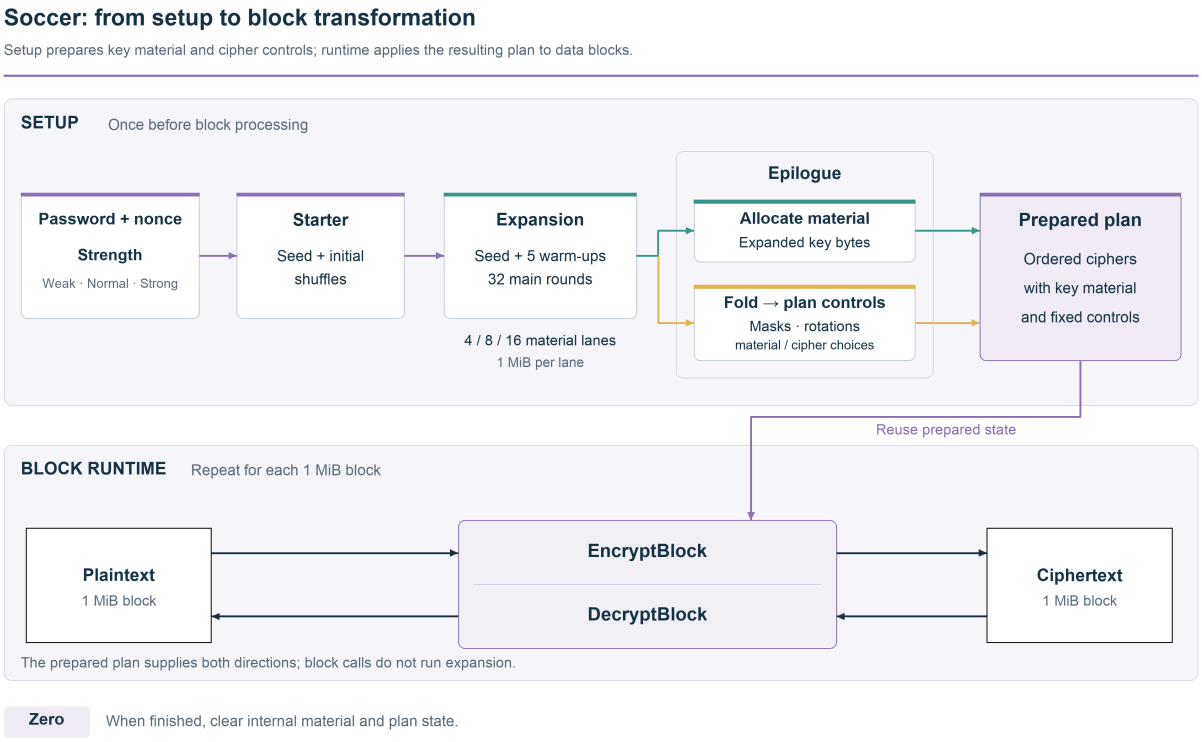


Figure 1: Soccer system overview. Stateful setup prepares the material and cipher plan; each 1 MiB data block is then processed without dependence on earlier blocks.

Material lanes and expansion

Soccer's material-expansion stage is intended to supply cryptographically secure pseudorandom material: the backbone from which both key bytes and encryption controls are derived. A **material lane** is a 1 MiB buffer assembled from 32 pieces of 32 KiB each. Weak, Normal, and Strong use 4, 8, and 16 active lanes, respectively, giving 4, 8, or 16 MiB of material. The lanes form an interacting stack: generated bytes influence later expansion and the eventual cipher plan. For a fixed password, nonce, and strength, setup is reproducible, allowing the same plan to be reconstructed for decryption. The security assumption behind this material is discussed in the expander construction and security discussion.



Figure 2: Material-lane construction. Each active lane receives a seed at slot 26, five warm-up pieces at slots 27-31, and 32 main-round pieces at slots 0-31. The main rounds overwrite the six preliminary pieces. All slot indices are zero-based.

Starter expander. Setup begins with the dedicated Betelgeuse starter. It processes a 32 KiB scratch input made from the password and nonce, with the nonce also supplied separately to its seed operation. Its 32 KiB output drives the initial shuffles of the expander list, workspaces, material buffers, and cipher list. This starter output establishes the initial assignments; the per-lane seed pieces shown in Figure 2 are produced afterward.

Strength and expander selection. A pool of 16 named expanders supplies the regular expansion stage. Seeding proceeds in four groups, selecting 1, 2, or 4 previously unclaimed expanders, workspaces, and buffers per group for Weak, Normal, or Strong. Between groups, the latest seed output reshuffles the remaining choices. Subsequent rounds make 4, 8, or 16 expander calls, one per active destination. The full expander list continues to be shuffled, so an active material lane has no permanent expander identity. Strength therefore sets the breadth of expansion, while generated material determines the evolving assignments.

Strength, data scales, and lane interaction

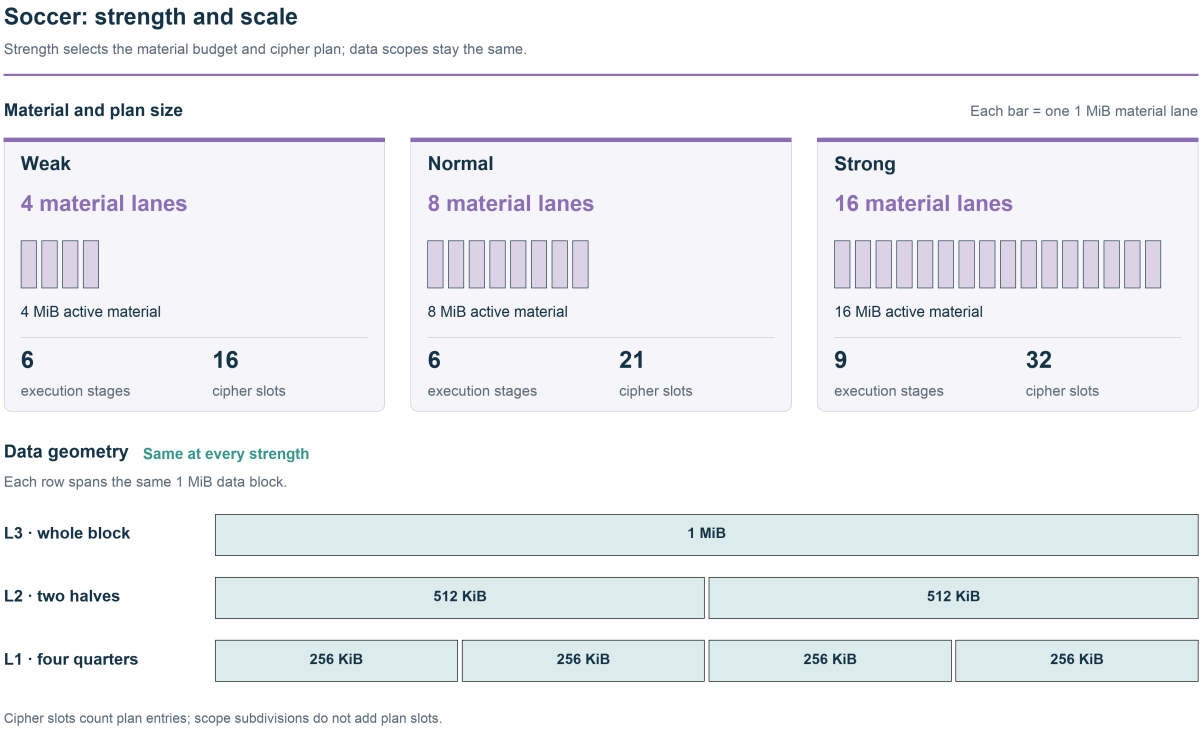


Figure 3: Strength changes the active material budget and the number of stages and cipher slots. Whole-block, half-block, and quarter-block scopes always cover the same 1 MiB data block.

Fixed data geometry. L3 operates on the whole 1 MiB block; L2 applies a layer to each 512 KiB half; L1 applies a layer to each 256 KiB quarter. Strength changes the material budget and recipe length while preserving these scopes. Cipher-slot counts describe configured plan entries; reusing a layer on each half or quarter does not add plan entries.

Nonce lanes. Each seeded workspace has a 32 KiB nonce lane. Three 64-bit diffusion functions transform the setup nonce, and their 24 output bytes repeat to fill this lane. Seed loops combine it with the password-derived source lane. Its role is to carry nonce influence through initialization; its length does not represent fresh entropy. The nonce is treated as public input in the security analysis.

Four cross inputs. Each expansion call receives a current source and four ordered references to earlier 32 KiB pieces. During main rounds, a material-controlled permutation chooses four distinct entries from a pool of 16, 32, or 64 candidates. A pool combines other current sources with older pieces spanning the preceding five output positions; several selected pieces may belong to the same physical buffer. The first warm-up supplements the seeds with starter output, and later warm-ups use the growing history.

Warm-up and main rounds. One seed occupies slot 26; five warm-ups fill slots 27-31. The 32 main rounds then fill slots 0-31, overwriting that initial history in rounds 27-32. Calls run sequentially with shared mutable ARX state. After each round, new pieces become sources, folding across lanes produces 32 KiB of shuffle controls, selected pieces are reversed, and assignments are reshuffled. These setup rounds precede the data-encryption stages.

Four final summary lanes. Weak feeds its four material lanes directly into folding; Normal and Strong first collapse their stacks to four 1 MiB summaries. Each summary is reduced through 256 KiB and 64 KiB to 16 KiB. Summaries A and B supply values for a 2 KiB control seed; C and D control their read order, seed shuffling, and further material and mask shuffles. These results determine rotations, masks, and plan choices. The full material stack remains available for the cipher key slices described below.

Cipher selection and fixed role recipes

Development began with 38 candidate cipher primitives, screened for their roles and performance in optimized SIMD and Arm NEON code. The current regular composer retains 21 types in a weighted 256-entry pool. It builds a Cryptex from a fixed sequence of roles, filling each slot by scanning the material-shuffled pool for a compatible type.



Figure 4: A concrete Normal plan with 21 slots. Roles stay fixed; the selected cipher types and their parameters depend on setup material. This example uses an illustrative pool permutation and lane-combination row 0.

Rotation (R or RM) is a required organizing operation at the start of every stage. Plain rotation moves all bits; masked rotation moves selected bit positions. Offsets come from folded setup material. Intervening mutations and movements separate rotations, so their effects must be assessed as part of the complete composition, including cases with complementary offsets.

Primary movers (P) provide broad rearrangements, including masked reversal and splint operations. **Secondary movers (Q)** supply additional patterns, including ripple, byte-block reversal, and masked inversion. These are the implementation's role categories. **Password-style mutations (S or C)** alter values using expanded key-material slices, with simple and complex eligibility flags. Here, "password" names the input role of derived material; runtime ciphers do not read the user's password directly.

The scan enforces both role eligibility and the lane plan's material-input count. It advances a shared cursor through the pool and may reuse a cipher type. Thus the role recipe is fixed while compatible occupants, masks, rotations, and key slices vary with setup.

Bounding slower operations. Development timing records identify `CascadeCipher` and `CascadeJumpCipher` as the slowest retained primitives. Each has three entries in the pool. More decisively, both require a complex mutation slot: Weak and Normal have none, while Strong has four, bounding Cascade-family occupancy at four plan slots. Other eligible mutations can fill those slots when their material-input counts match. This constrains cost while retaining the slower variants where the recipe permits them.

Allocating the material lanes

A lane plan assigns the number of material inputs required by each mutation slot. Each input has the size of its stage: 1 MiB at L3, 512 KiB at L2, or 256 KiB at L1. The composer allocates all L3 requests first, then L2, then L1, packing smaller requests into the remaining space. This construction order prevents gaps and is distinct from Cryptex's execution order.

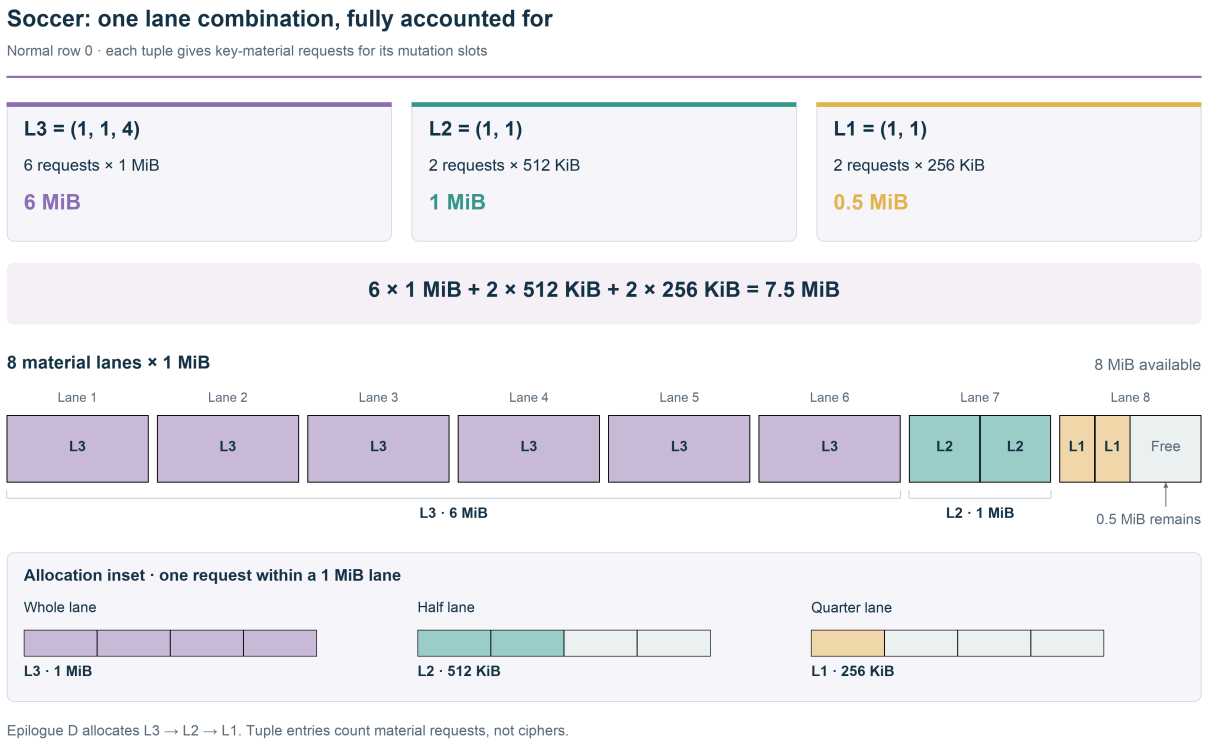


Figure 5: Normal lane-combination row 0 uses 7.5 MiB: six full-lane inputs, two half-lane inputs, and two quarter-lane inputs. All eight material lanes contribute, with the final half-lane left unallocated to cipher keys.

The complete tables allow the following totals, measured in 1 MiB lane units:

Strength	Active lanes	Permitted key-material allocation
Weak	4	3.25, 3.50, 3.75, or 4.00 lanes
Normal	8	7.25, 7.50, 7.75, or 8.00 lanes
Strong	16	15.25, 15.50, 15.75, or 16.00 lanes

Every plan reaches the final active lane and consumes one, two, three, or four of its quarters. In particular, **Weak can use 3.25 lanes**; it does not always allocate all 4 MiB to ciphers. One such row is L3 = (1, 1), L2 = (1), L1 = (3): its demand is 2 + 0.5 + 0.75 = 3.25 MiB. Across all 12 Weak rows, the four totals occur 2, 2, 4, and 4 times, respectively.

In Figure 5, Normal's L3 tuple (1, 1, 4) requests six full-size inputs, and the smaller stages use another 1.5 MiB. The remaining 0.5 MiB has already participated in expansion and folding, even though it is not bound to a cipher as key bytes. Material lanes are therefore all used in setup, and all are also touched by the subsequent key allocation.

A mutation requesting several inputs consumes several stage-sized slices. Rotations and movers instead take compact controls such as offsets and masks; they do not consume these bulk key-material slices. Applying an L2 or L1 layer to multiple data slices reuses its configured key material, so it does not multiply the allocation shown here.

Mask packs and selective bit movement

Masks determine which bit positions participate in a masked operation. Setup selects one of 420 stored packs, each containing 13 eight-bit masks, then shuffles the pack using derived controls. Each mask has exactly four set bits. Across a complete pack, each bit position is selected six or seven times, providing balanced coverage of the eight positions at the pack level.

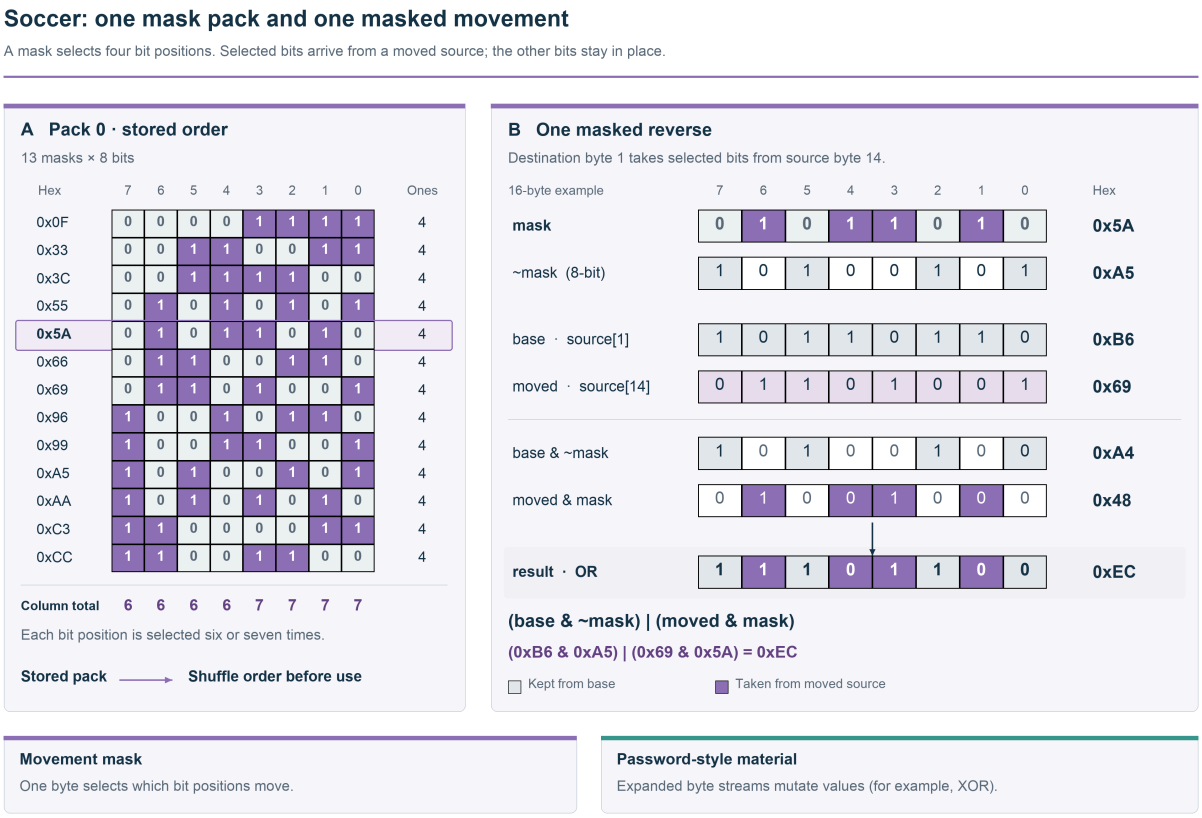


Figure 6: One stored mask pack and a masked-reversal example. Mask 0x5A selects four bit positions; the unselected bits stay at the base position while selected bits arrive from the reversed source position.

For a source array of length n, masked reversal constructs destination byte i from the base byte at i and the moved byte at $n - 1 - i$:

```
destination = (base & ~mask) | (moved & mask)
```

The complement is restricted to eight bits. In Figure 6, the mask is `0x5A`, its complement is `0xA5`, and the two source bytes are `0xB6` and `0x69`. The retained contribution is `0xA4`; the moved contribution is `0x48`; their bitwise OR gives `0xEC`. The 16-byte array illustrates a primitive operation; production layers use the larger L3, L2, and L1 scopes.

The mask cursor supplies entries cyclically from the shuffled 13-mask pack when the composer instantiates masked operations. Consequently, the pack's six-or-seven balance describes the complete pack, not a promise about every short prefix of masks used by a plan. The diagram shows stored pack order; the actual order is material-dependent.

A movement mask is a compact selector for bit positions. A password-style cipher receives expanded byte streams that alter data values or govern key access. Both inputs are derived during setup, but their sizes and functions are different. Together with rotation and the fixed role recipes, masked movement gives the composition repeated opportunities to rearrange selected bits between material-driven mutations.

A layer as a composite cipher

A configured cipher provides a forward transformation and a matching inverse. Its key material and controls fix both directions. If a cipher maps input x to output y, applying its inverse to y recovers x. Some operations, such as XOR with the same key, undo themselves. Others require a different inverse operation: a right rotation is undone by a left rotation of the same amount, and byte addition is undone by subtraction modulo 256.

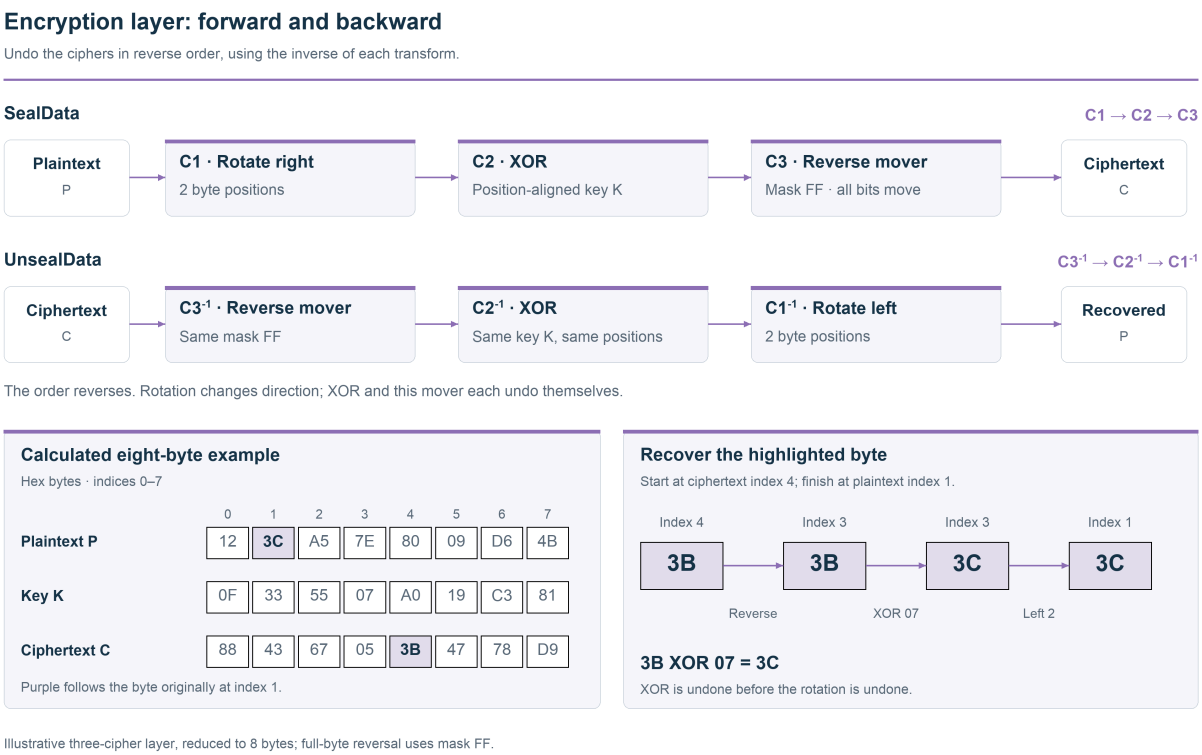


Figure 7: A layer combines three reversible operations. Encryption applies C1, C2, then C3; decryption applies the inverse of C3, then C2, then C1. The small byte example illustrates the same ordering.

An **encryption layer is itself a composite cipher**: it accepts one data slice, passes the result through several constituent ciphers, and returns one slice of the same length. With three configured ciphers, its forward map is

$$L(x) = C_3(C_2(C_1(x))).$$

Its inverse is obtained by undoing the last transformation first:

$$L^{-1}(y) = C_1^{-1}(C_2^{-1}(C_3^{-1}(y))).$$

The rightmost function acts first in these expressions. Consequently, $L^{-1}(L(x)) = x$ when the matching key material and controls are used. `EncryptionLayer` implements this directly: `SealData` visits its cipher list from first to last; `UnsealData` visits it from last to first and invokes each cipher's inverse. Reversing the list alone is insufficient when a primitive's forward and inverse operations differ.

In Figure 7, a byte starting at index 1 moves to index 3, is XORed with the key byte at that position, then moves to index 4. Decryption reverses those steps. The key stays aligned to the current data positions; it does not travel with the highlighted byte. The eight-byte geometry and full-reversal mask `0xFF` are teaching choices. Production layers use the L3/L2/L1 scopes, and the regular mask packs select four bit positions per mask.

This composition gives Cryptex a convenient building block: each layer can be applied and inverted as one transformation, even though it contains several cipher calls. Cryptex composes these layers at the whole-block, half-block, and quarter-block scopes in the next section.

Cryptex execution and inverse order

The prepared Cryptex binds the selected ciphers to key slices, masks, and rotation offsets. Weak and Normal use Cryptex6; Strong uses Cryptex9, adding an L3C, L2C, L1C group after the A and B groups.

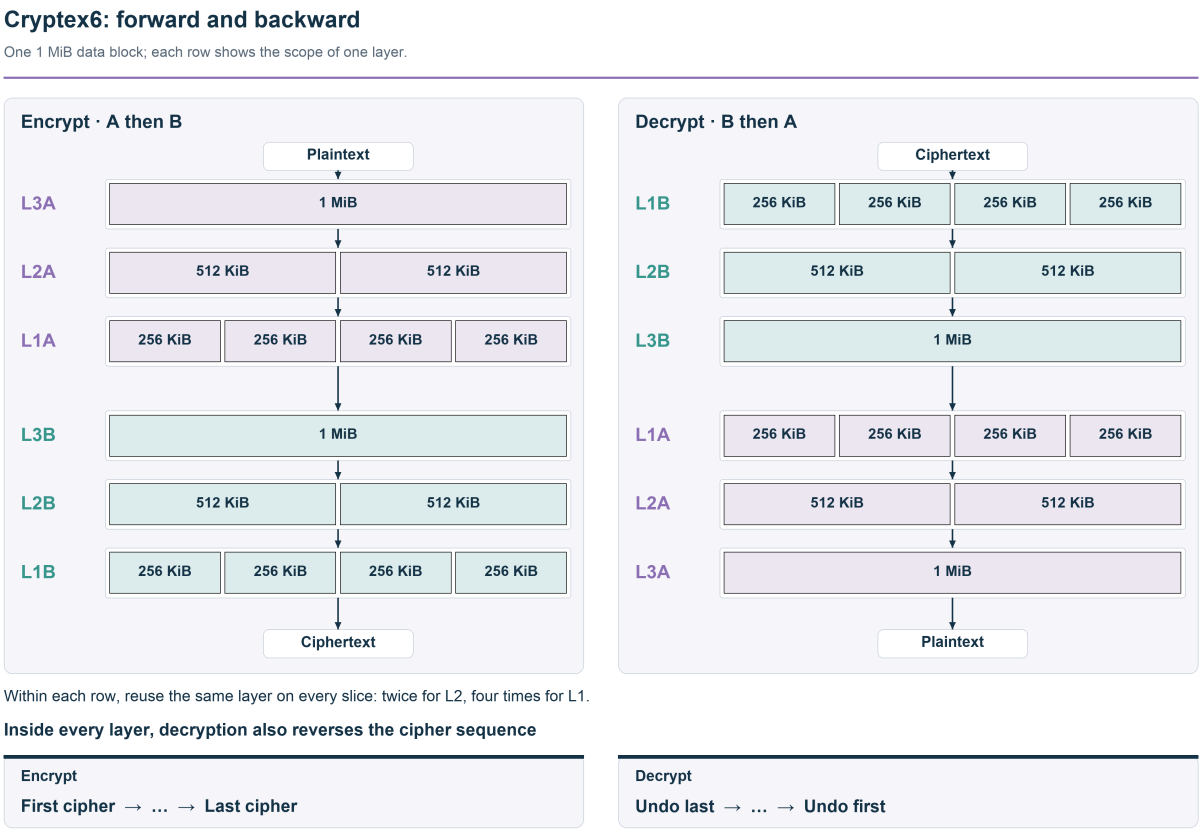


Figure 8: Cryptex6 processes one 1 MiB block through six stages. Decryption reverses the stage order and applies each stage's ciphers in reverse order using their inverse operations.

At L2, the same configured layer processes both halves; at L1, it processes all four quarters. Decryption reverses the order of layers and ciphers, while the disjoint halves or quarters can still be visited in ascending address order. Each 1 MiB input uses the prepared plan independently.

The 1 MiB block size was chosen to support efficient bulk SIMD and Arm NEON processing. Its 512 KiB and 256 KiB subdivisions are also exact multiples of the 16-, 32-, and 64-byte granularities used by the primitives. These fixed sizes support regular vector loops and bulk movement. Development observations motivated the fast retained set, including the comparatively slower Cascade variants; reproducible throughput figures require a stated processor, compiler, and benchmark workload.

The following sections describe the machinery that supplies the plan: the six functional domains, M88 byte-permutation diffusion, and Aldebaran as a representative expander. The walkthrough distinguishes workspace state from returned material and explains how a workspace can continue under a different regular twister. Its security assumption remains an explicit hypothesis for subsequent analysis.

Functional domains

A **domain** is a named bundle of derived material assigned to one part of the expander. The workspace retains six domains: KeyRotate A, KeyRotate B, KeySpawn A, KeySpawn B, Twist, and Seed. The two KeyRotate domains govern the replacement of key-box rows; the two KeySpawn domains govern their initial construction. Twist supplies the controls for subsequent output calls, and Seed supplies those for the main initialization path. This separates the material used by different functions while keeping a common derivation structure.

Each domain contains three salt records and eleven control constants. Each salt record carries 32 KiB, giving 96 KiB of salt payload per domain. The constants comprise five 64-bit words and six bytes, used for ingress, scatter, cross-input processing, matrix selection, and matrix arguments. These sizes describe stored payload, not additional entropy or the padded size of a C++ structure. The salt records are internal working material derived during setup.

The setup order in Figure 9 follows the implementation. Starting from the selected expander's built-in material, each domain passes through three KDF stages, each followed by Farm. The first two results populate temporary bundles; the third supplies the domain retained in the workspace. Farm copies three output lanes into salt records and folds a fourth lane into constants. The password-derived source, nonce, and base ARX state are shared inputs, while domain-specific material and generated routines distinguish the use sites. ARX state continues across the three passes within a domain and is reset to the captured base when the next domain begins.

This organization makes the consumers of each bundle explicit. Naming and separating fields alone does not establish cryptographic independence between their contents. The Aldebaran walkthrough later expands the Seed-domain example in Figure 16.

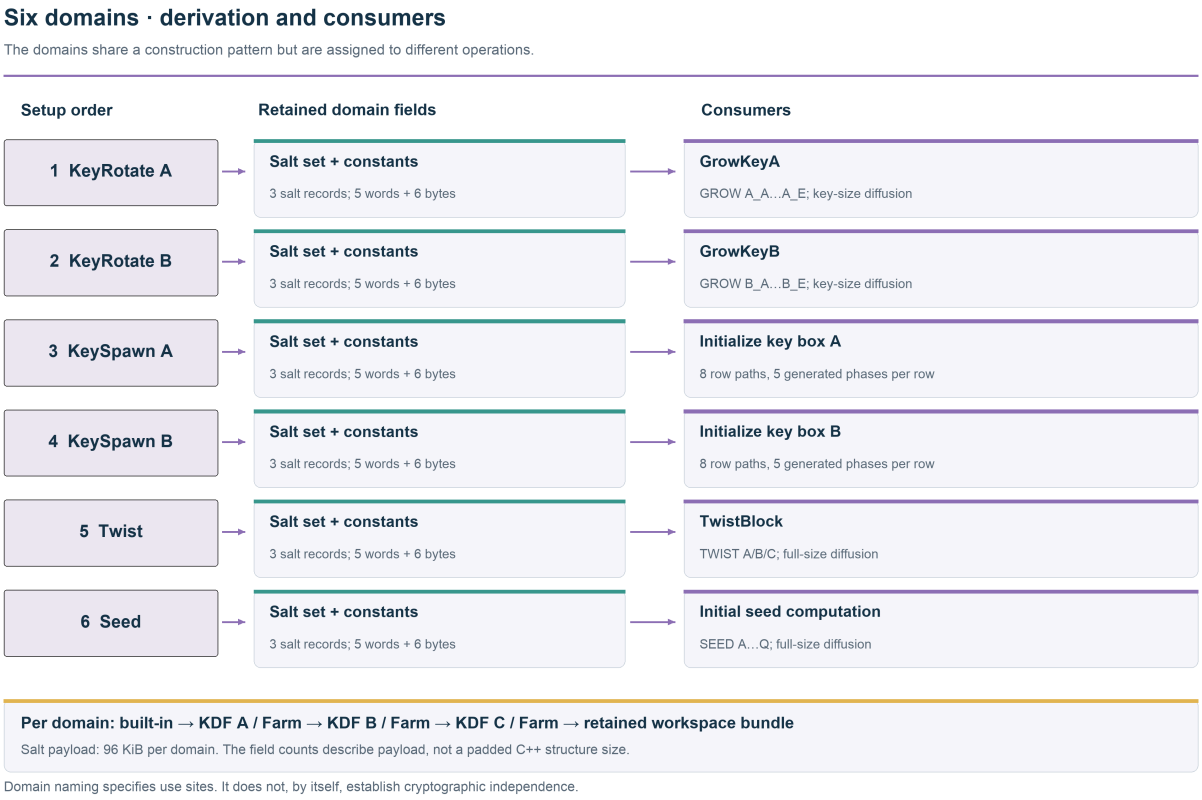


Figure 9: Six domains in setup order, with their retained fields and consumers. Each follows the same three-pass KDF/Farm pattern.

Matrix diffusion by byte permutation

The expander deliberately assigns different jobs to arithmetic and matrix processing. Generated **ARX loops** combine addition, rotation, and XOR to change values and produce the intended pseudorandom material. **Matrix calls** redistribute those values across positions and lanes before later arithmetic uses them again. M88 represents a 64-byte tile as an 8×8 array, with four 4×4 quarters and sixteen 2×2 minis. Every cell is one byte.

The matrix stage intentionally uses byte permutations instead of value-mixing matrix operations such as AES MixColumns, which combines the bytes of each column using a fixed finite-field matrix (FIPS 197, Section 5.1.3). For fixed controls, an M88 tile operation rearranges the input bytes and preserves their values and multiplicities. Thus "diffusion" here identifies spatial redistribution within the larger ARX-and-permutation construction. A permutation alone does not create entropy or make one changed byte become several changed bytes; propagation across values depends on the surrounding ARX computation and the dependence of controls on prior material.

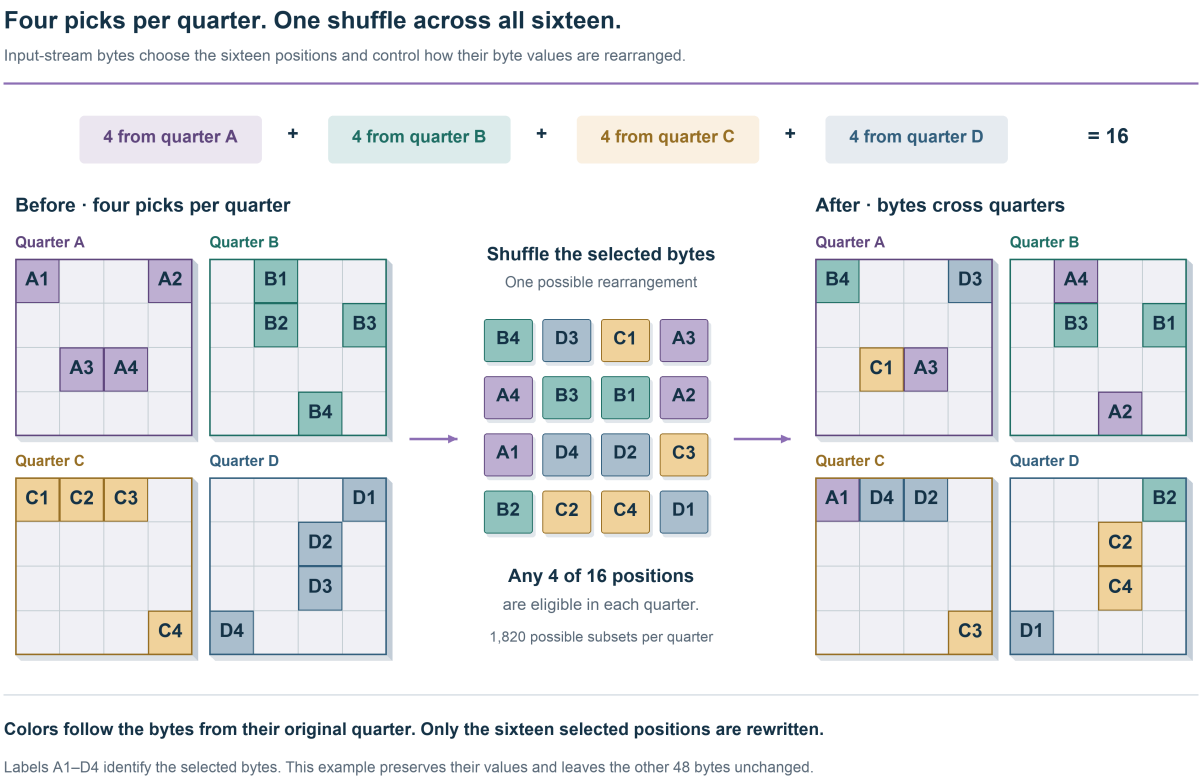


Figure 10: An active selected-cell permutation. Four positions are chosen from each quarter, and the sixteen selected bytes are rearranged across the combined set. The other 48 positions retain their values.

The selector can reach every four-position subset of a quarter: there are $\binom{16}{4} = 1820$ possibilities. Two control bytes per quarter select a table entry. Further control bytes reorder the combined position list and choose its rotation. The figure shows an active, nonzero case; a zero rotation amount bypasses this step. Selection and rearrangement are deterministic functions of the input streams. Reachability of every subset does not imply uniform probabilities or access to all 16! rearrangements in one call.

A complete matrix call and one tile's path

A full-size diffusion call accepts four 32 KiB data lanes and four control lanes, and writes four 32 KiB result lanes. Each data lane contains 512 64-byte tiles. Control-derived index lists and lane order route source tiles to output positions; each selected tile then undergoes an M88 dispatch. The key-size variant uses the corresponding smaller 2 KiB spans.

Within a tile, dispatch performs four passes. Each pass applies the selected-cell permutation, full operations A and B, an operation in each of the four quarters, one selected mini operation, and controlled row, column, and mini swaps. Final row selection serializes the transformed tile. The passes reuse overlapping control bytes together with domain arguments; the choices are not modeled as independent random draws.

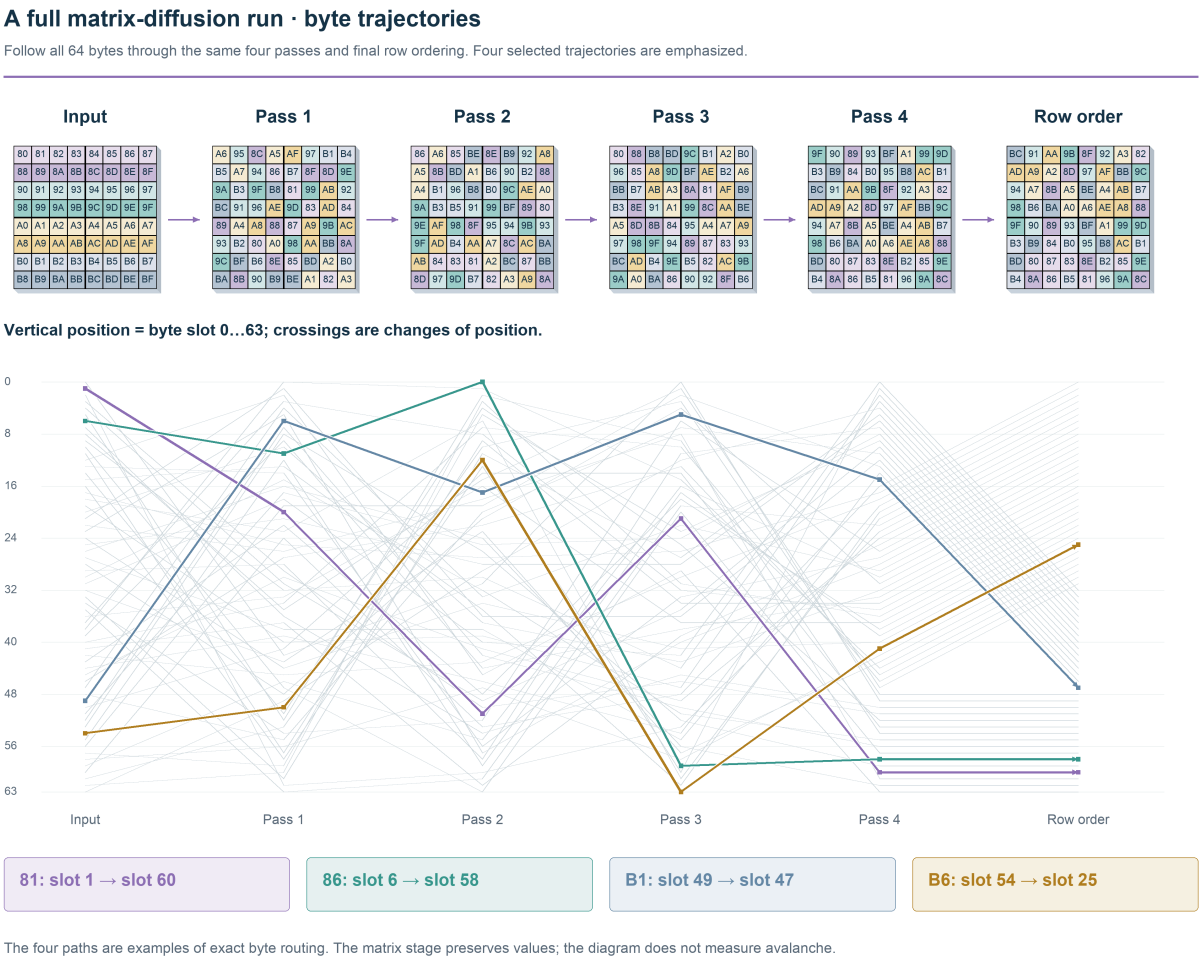


Figure 11: One 64-byte tile through all four M88 passes and final row ordering. The small grids and trajectories describe the same native implementation trace; four byte paths are emphasized.

The full-call routing operates across lanes, while Figure 11 expands one tile's complete transformation. Its changing colors and trajectories show where bytes travel, not changes in byte values. The labels preserve provenance through every pass. This fixture demonstrates the implemented routing; it is not an avalanche measurement or a claim about the distribution of outputs.

Full operations and structural separation

Many operations were selected partly for human-readable aesthetics: visible rotations, folds, interleavings, and structured rearrangements make the maps easier to inspect. This was an intentional design choice. The full catalogue contains 512 distinct maps across the two 256-entry dispatch tables. A full operation acts on the 8×8 tile, although it need not move every cell.

For two maps p and q, define their **positional difference** as the number of output positions receiving different source-byte identities: $d(p, q) = |\{i : p(i) \neq q(i)\}|$. Exhaustive pairwise comparison of the full catalogue gives **minimum 24 and maximum 64**. These distances distinguish maps; they are not bit distances, movement counts relative to the input, or a security bound.

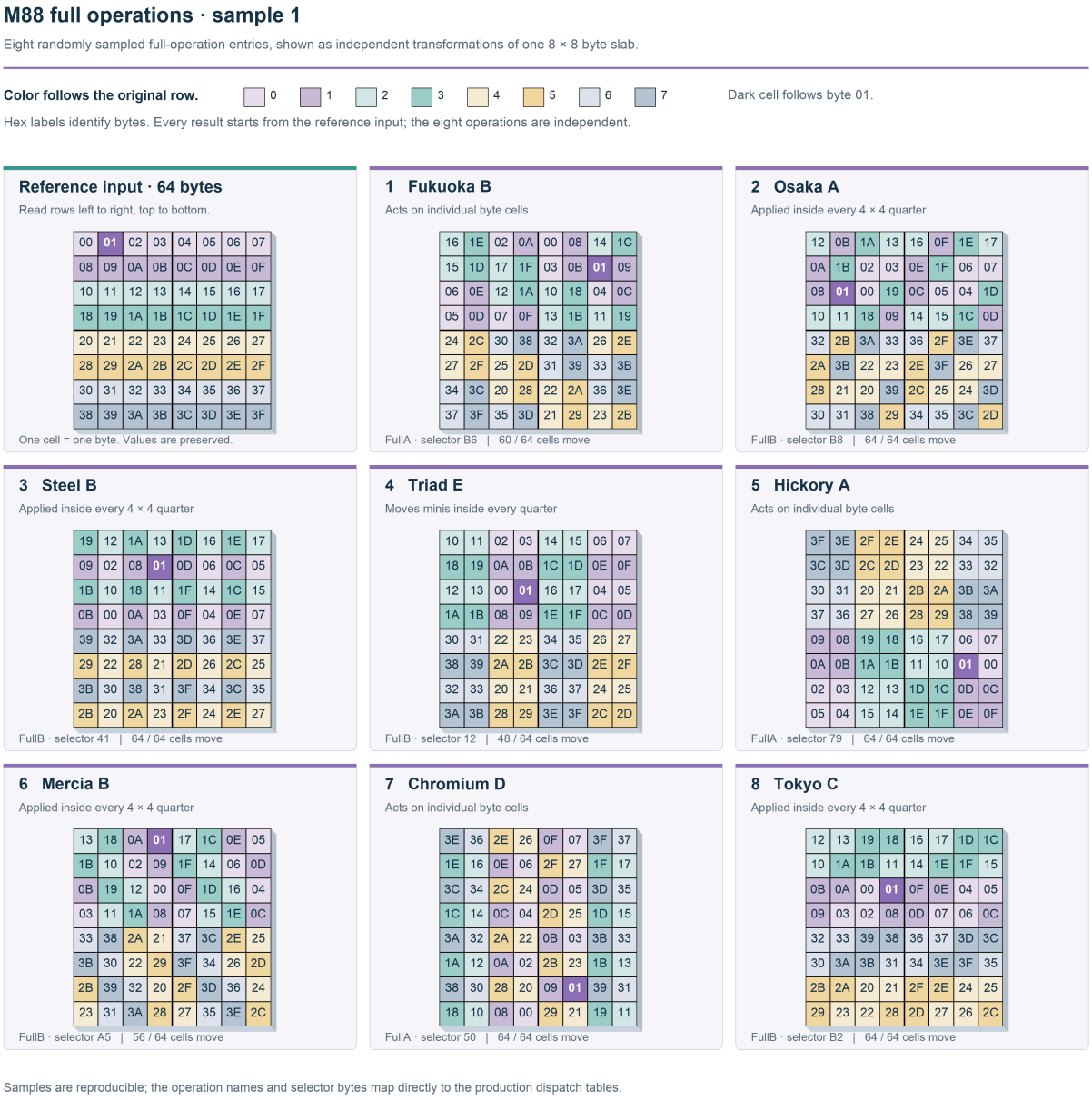


Figure 12: Eight independent full-operation examples. Each starts from the same 64 distinct byte identities; colors follow original rows and the highlighted cell follows byte 01.

Quarter operations

A quarter operation is confined to one 4×4 region. Its sixteen bytes can be rearranged individually, moved as 2×2 minis, or transformed within each mini. The other 48 bytes of the surrounding tile stay in place. Quarter A is upper-left, B upper-right, C lower-left, and D lower-right. Each location has a 256-entry dispatch table.

Comparing distinct operations **at the same quarter location**, using the positional difference defined above, gives **minimum 8 and maximum 16**. These bounds hold for each of the four quarter tables. Fixing the location is essential: comparing operations in two different quarters also measures the change of support. The enlarged results below preserve the original 64-byte tile's labels, so movement can be checked against the reference.

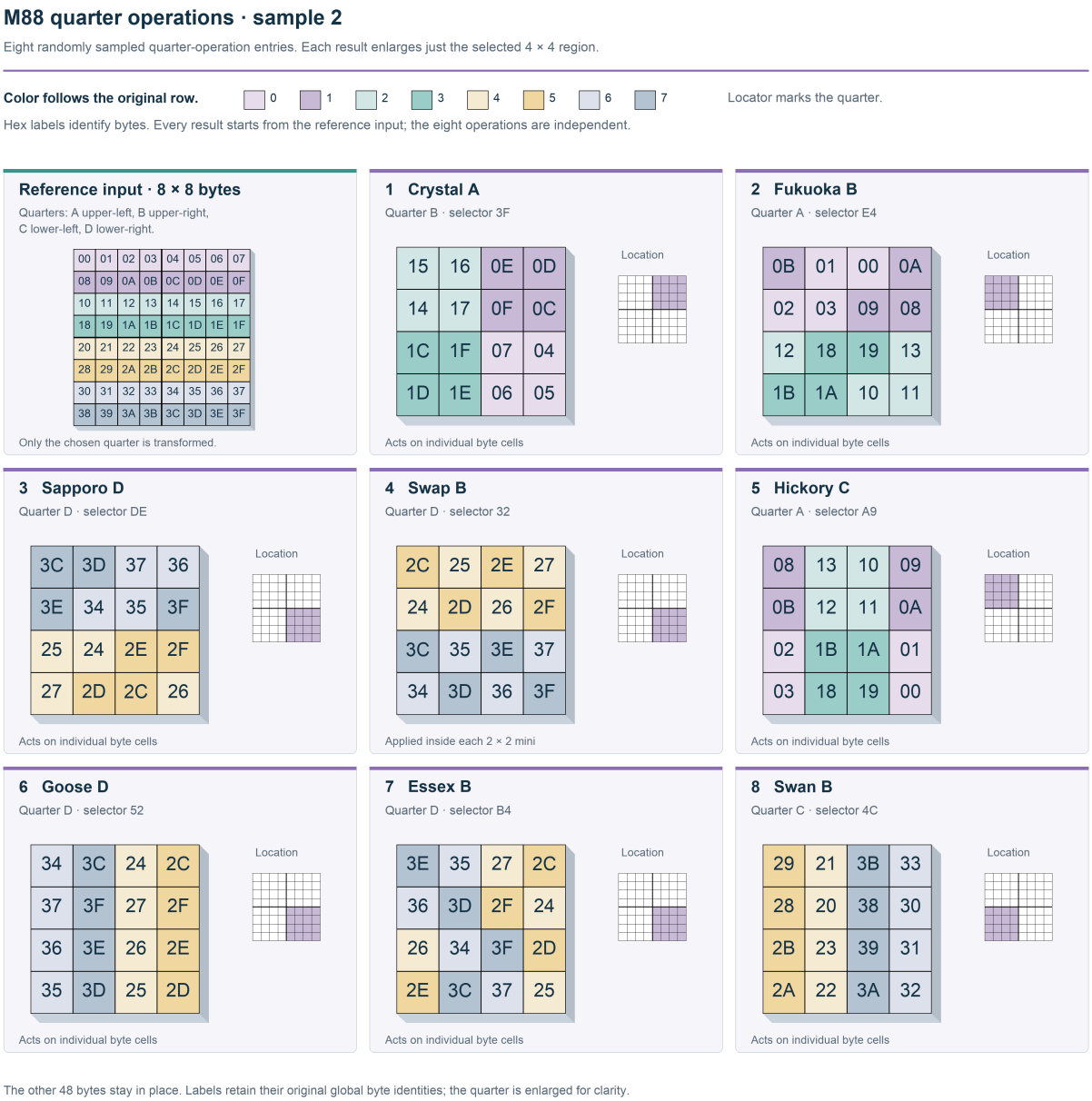


Figure 13: Eight independent quarter-operation examples. Locator panels identify the active quarter, which is enlarged to expose the byte rearrangement.

Mini operations

A mini operation acts on four bytes arranged as a 2×2 square. The selector's high nibble chooses one of sixteen operations, and its low nibble chooses one of sixteen mini locations in the tile. The other 60 byte positions are unchanged by that operation. This gives local rearrangement between the broader full and quarter transformations.

For the sixteen distinct operations **at a fixed mini location**, the minimum pairwise positional difference is **2**, and the maximum is **4**. The full, quarter, and mini ranges are therefore **24-64**, **8-16**, and **2-4**, respectively. They describe separation between catalogue maps on distinct byte labels. A visually regular pattern or a large positional distance does not, by itself, establish the cryptographic quality of the complete expander.

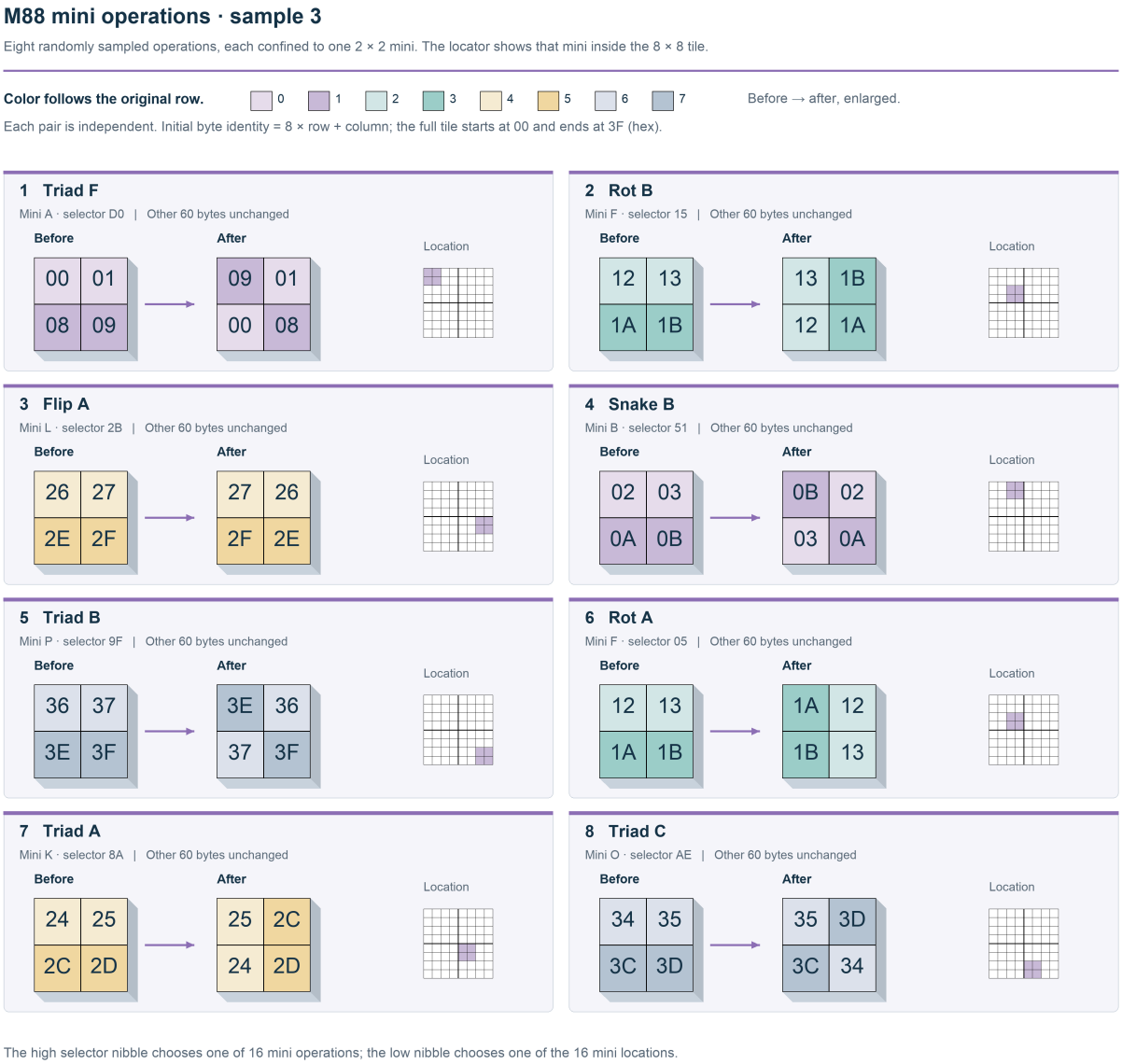


Figure 14: Eight independent mini-operation examples, each shown before and after. Locator panels identify the selected 2 by 2 region inside the full tile.

Aldebaran: a representative expander

Aldebaran provides a concrete walkthrough of the common TwistExpander interface. **Seed** initializes a workspace from password and nonce inputs, derives six domains, produces an initial 32 KiB material piece, and fills two key boxes. Later **TwistBlock** calls accept a source piece and four cross inputs, write another 32 KiB piece, and advance the state. These output pieces are assembled into the larger material lanes described earlier.

The workspace stores derived domains, key boxes, and scratch buffers. Carried ARX parameters are passed separately: ingress, carry, and eleven 64-bit wanderers. Internal groups such as Crystal A-D are four working lanes; the returned 32 KiB destination is a fold of that group. It is distinct from both the workspace and the composer's 1 MiB material lane.

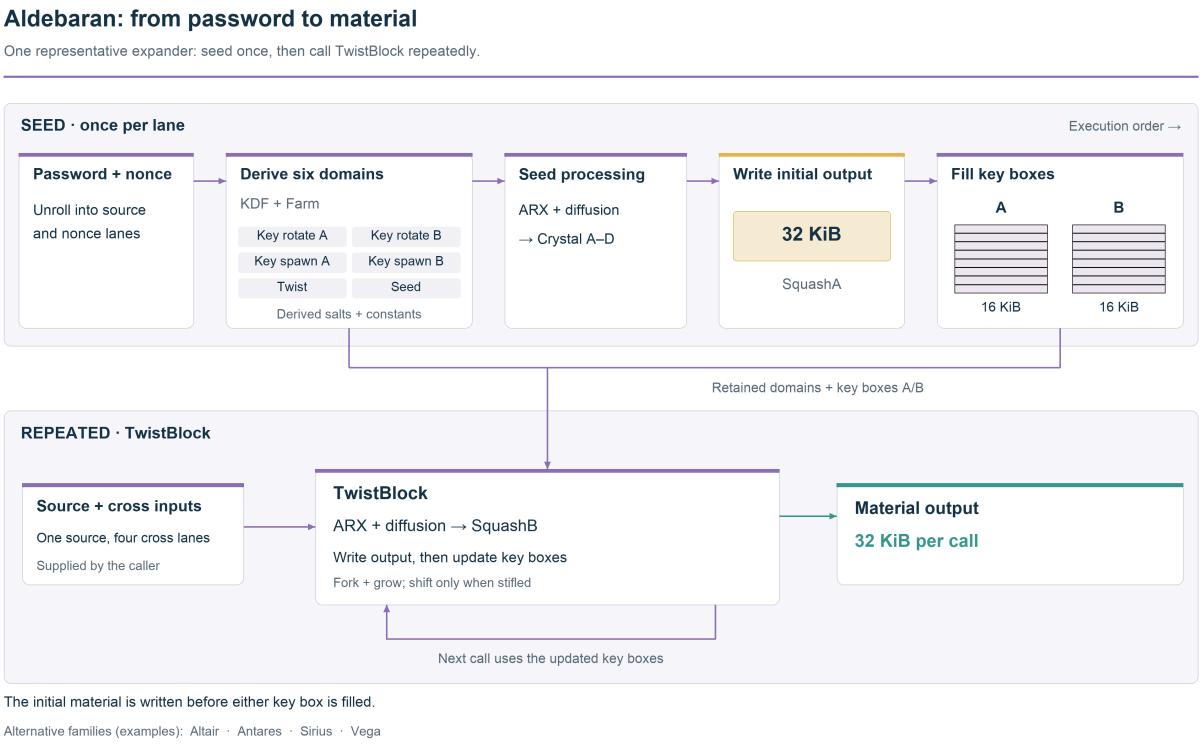


Figure 15: Aldebaran's initialization and repeated-output interface. Seed writes initial material before filling the key boxes; subsequent calls write a destination block before maintaining those boxes.

Security assumption and scope. The security foundation is the assumption that the password-and-nonce-driven expanders supply cryptographically secure pseudorandom material. The proposed password-recovery resistance objective is computational infeasibility of recovering the password from ciphertext or exposed final material, even with the public nonce known. Repeated matrix rounds motivate this objective but do not prove it. Offline guessing remains possible; its cost depends on password entropy and setup work, as discussed in NIST SP 800-132, Appendix A.1. The walkthrough specifies calls and data flow. The closing security discussion separates output prediction, state exposure, and fixed-plan leakage; these remain obligations for independent analysis.

Aldebaran's domain-derivation chain

The Seed domain illustrates the three-pass construction used for all six roles. Seed first expands the password into a 32 KiB Source lane and the 64-bit nonce into a 32 KiB Nonce lane. The nonce scalar is also passed to the KDF routines. Expansion repeats prepared input material; these lane lengths do not represent additional entropy.

The first Seed-domain pass, `KDF_A_F`, uses built-in Seed salts and constants and leaves four Kinetic lanes. Farm takes Kinetic B, C, and D as the three salt sources and folds Kinetic A into the next constants. This populates the Ephemeral A bundle. The next pass, `KDF_B_F`, produces Abjuration A-D and feeds an equivalent Farm into Ephemeral B. Finally, `KDF_C_F` produces Crystal A-D; the last Farm writes the Seed fields retained in the workspace.

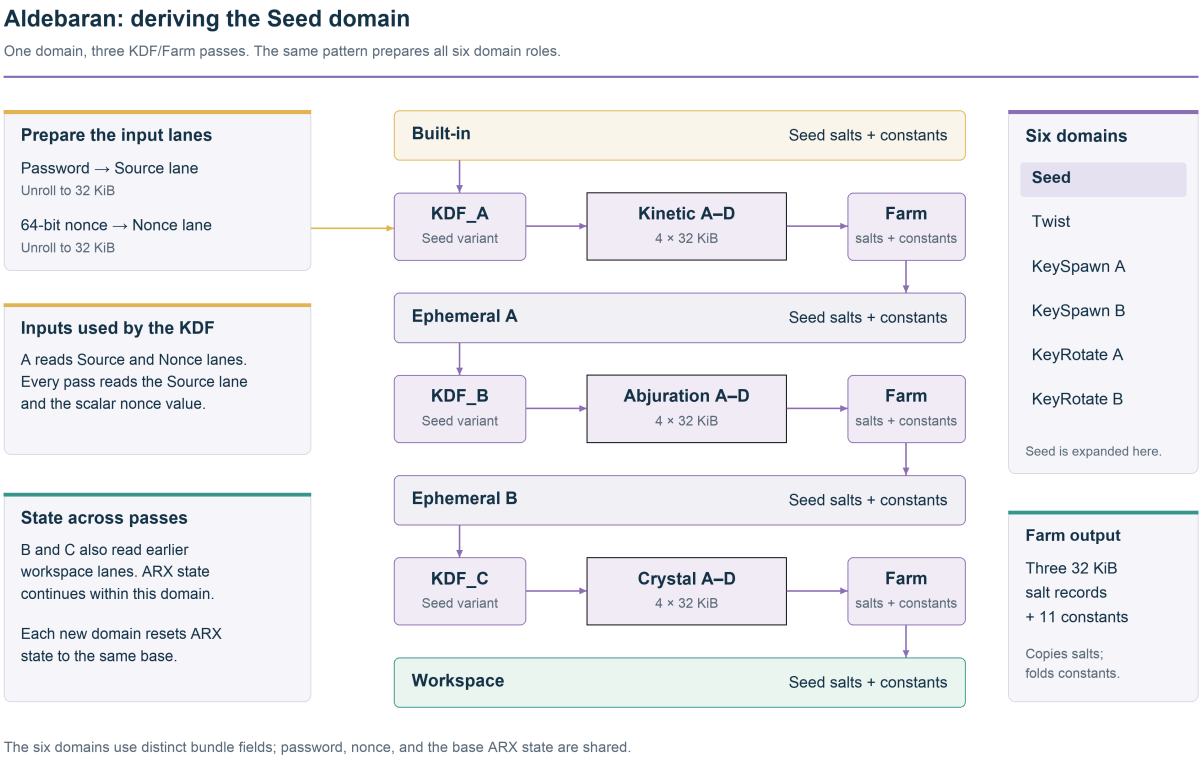


Figure 16: The Seed-domain chain in Aldebaran. Three KDF/Farm passes connect built-in material, two temporary bundles, and the retained workspace domain. The sidebar lists roles, not setup order.

Each Farm copies three 32 KiB salt records and derives eleven constants from the fourth lane. The constant fold reduces 32 KiB through 4 KiB and 512 bytes to a 128-byte prefix before selecting and deriving the retained fields. Those temporary prefixes reuse scratch storage. Subsequent KDF passes also read earlier workspace lanes, while mutable ARX state carries through the three passes. Starting another domain restores the common captured ARX base; it does not make the six resulting domains independently sampled randomness.

Seed once, reuse the workspace

A workspace is initialized by one chosen expander, but it is not permanently owned by that family. The regular expanders share the same workspace layout and domain roles. Consequently, **any of the compatible regular twisters can continue a workspace seeded by another**. The continuing twister applies its own generated routines to the retained domains and key-box contents. It need not repeat Seed or rebuild those structures merely because its identity changes.

The composer makes this interchange explicit: expander and workspace lists are shuffled separately, and each round calls the currently assigned expander with the currently assigned workspace. Source, cross inputs, and destination are also selected by the caller. Thus workspace history, expander identity, and output-lane identity are separate parts of the schedule. Figure Figure 17 follows one workspace to expose the local update rule; it does not require the same twister to execute every numbered call.

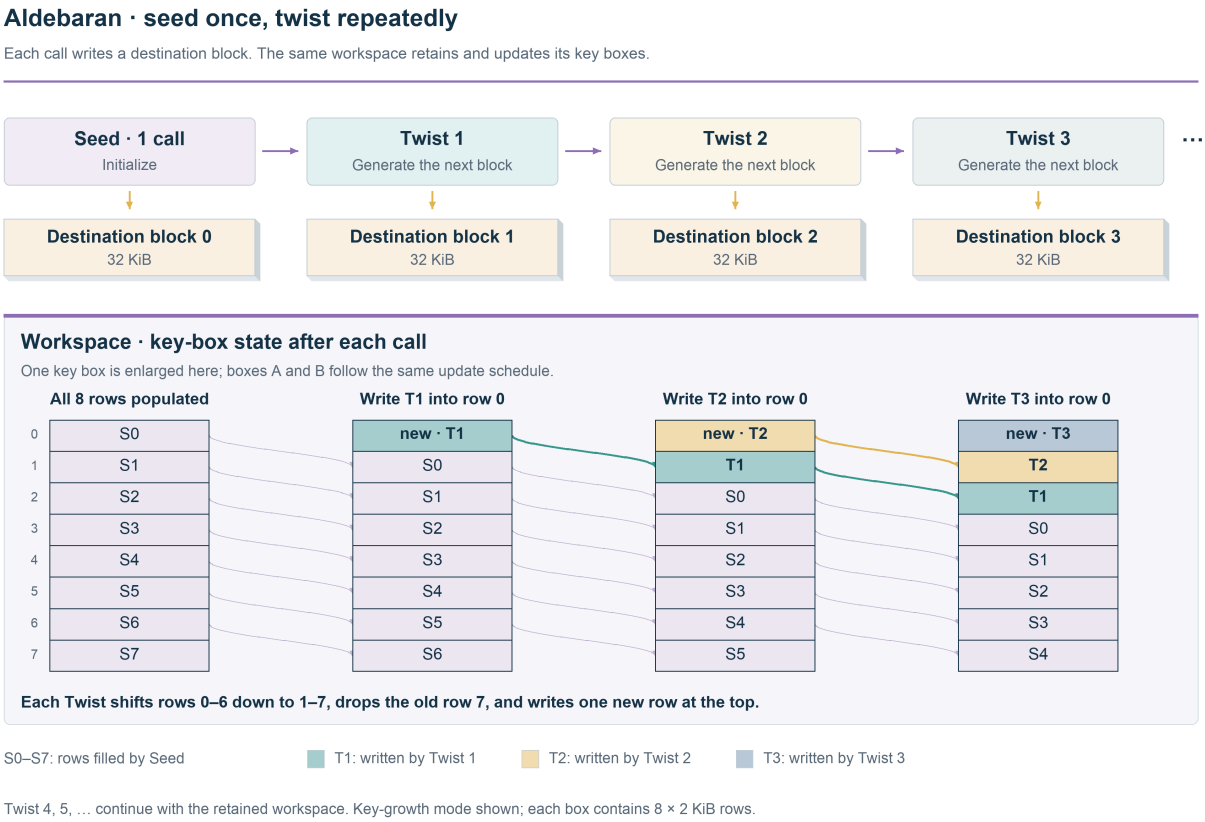


Figure 17: One Seed followed by three Twist calls in key-growth mode. Each call writes a 32 KiB destination. One of the two eight-row key boxes is expanded to show retained and newly inserted rows.

Both key boxes contain eight rows of 2 KiB each. In normal key-growth mode, each Twist consumes the oldest rows, shifts rows 0-6 into positions 1-7, and inserts one new row at position 0 in each box. After three calls, each box contains three newly generated rows. This is a shift with replacement; the discarded bottom row does not wrap to the top. The stifled branch, described with Twist below, shifts without generating replacements.

Call walkthrough: one KDF pass

The first pass for the Seed domain is a compact example of how ARX and matrix processing alternate. `KDF_A_F` contains **three generated ARX phases and one full-size diffusion call**. An ARX phase here means a generated four-lane function, not a single arithmetic instruction or one iteration of its loops. The four output lanes of each phase occupy 32 KiB each.

The first ARX phase writes Shadow A-D from prepared inputs and state. The second writes Fire A-D. Diffusion uses Fire as its data lanes and Shadow as its control lanes, producing Vapor A-D. The third ARX phase writes Kinetic A-D, reusing earlier workspace results as well as prepared input. The diagram shows the principal progression and the control bypass; its arrows do not claim to enumerate every read inside the generated routines.

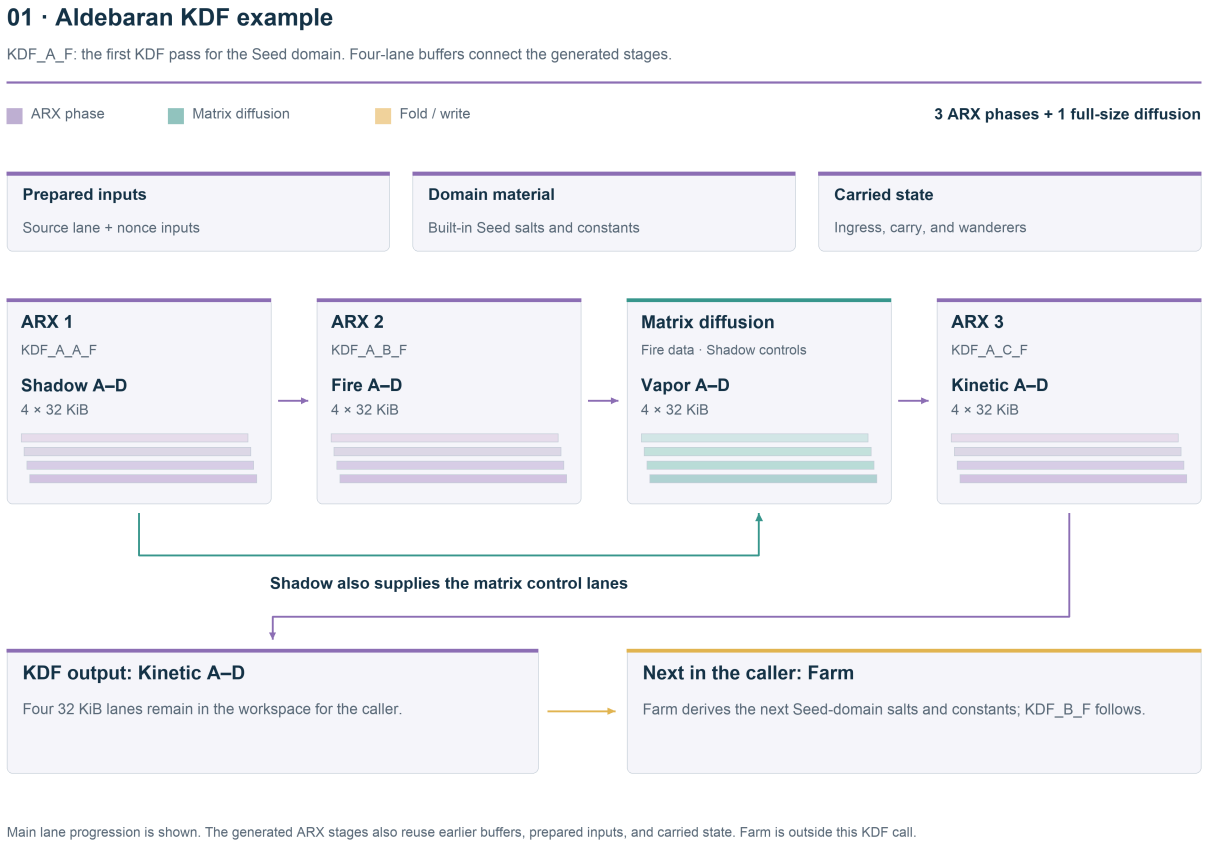


Figure 18: One KDF_A_F call. Shadow controls the rearrangement of Fire into Vapor, and a final ARX phase produces Kinetic. Farm belongs to the caller and follows this call.

The built-in Seed-domain salts and constants parameterize this first pass. Ingress, carry, and wanderers evolve through its phases and continue into the next pass. Kinetic is working material for Farm, rather than a returned data-encryption key or a destination block. Farm then derives the Ephemeral A Seed bundle, allowing `KDF_B_F` to continue the chain shown in Figure 16.

Call walkthrough: Seed

Seed performs the complete initialization of a workspace. After input preparation and the eighteen KDF/Farm passes that derive six domains, its main processing path has **17 generated ARX phases and eight full-size matrix diffusions**. Eight groups each execute two distinct ARX phases followed by a diffusion; a final ARX phase, `SEED_Q`, writes Crystal A-D. The grouped figure omits individual loop bodies while retaining that schedule.

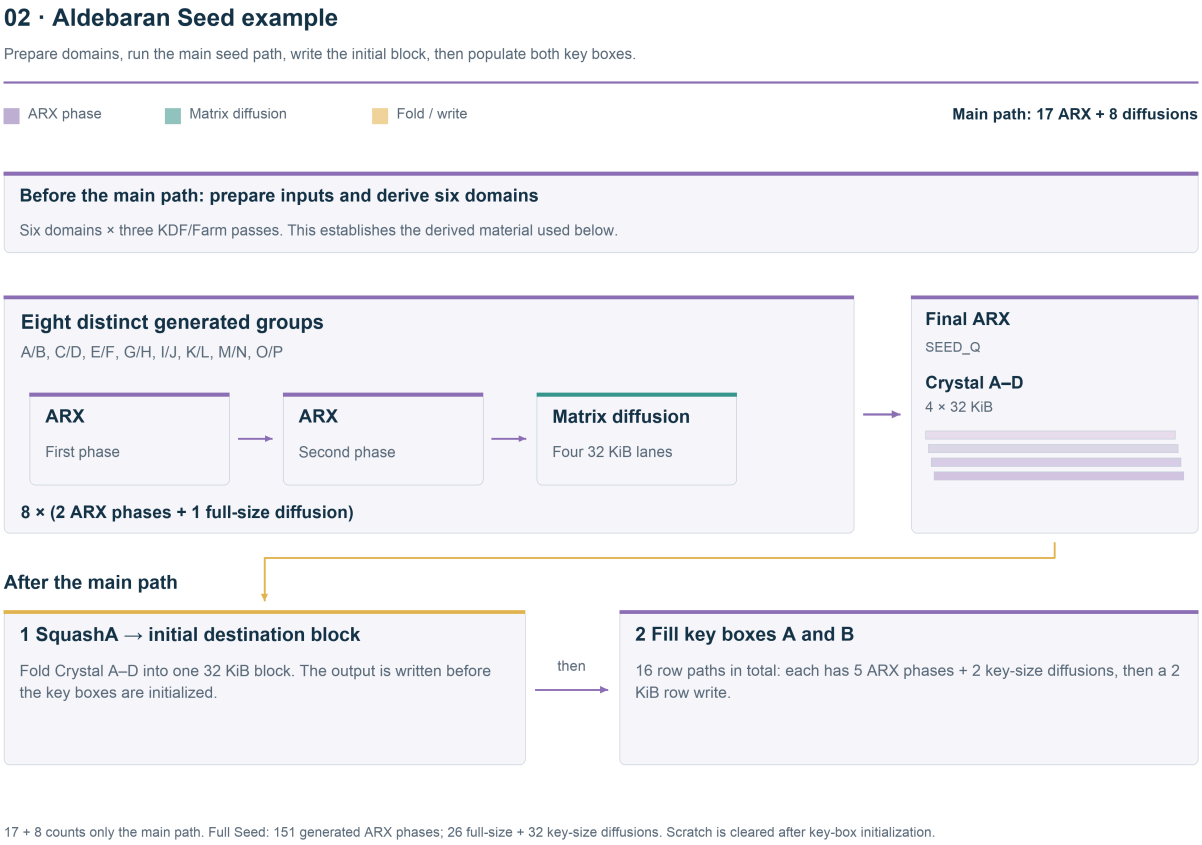


Figure 19: The main Seed path, followed by its two results. Eight two-ARX/diffusion groups and one final ARX phase precede the initial output write and key-box initialization.

`SquashA` folds Crystal A-D into the initial 32 KiB destination. Only after that write does Seed initialize key boxes A and B. Their sixteen row paths each execute five generated ARX phases and two key-size diffusions, followed by a fold into one 2 KiB row. This ordering distinguishes initial material production from the persistent key state prepared for subsequent Twist calls. Scratch is cleared after initialization.

The **17/8 count refers only to the main path**. Including nested domain setup adds 54 ARX phases and eighteen full-size diffusions; initializing the key boxes adds eighty ARX phases and 32 key-size diffusions. The complete Seed call therefore contains **151 generated ARX phases, 26 full-size diffusions, and 32 key-size diffusions**. Farm, output folds, copies, and scalar mixing helpers are separate work, not additional matrix stages in this count.

Call walkthrough: grow key box A

`GrowKeyA` creates one replacement row using the KeyRotate A domain. Before it runs, the caller invokes Fork A to derive Spirit A-D from retained Twist results. Grow also reads earlier full-size workspace buffers and the caller's cross inputs A and C. Its generated output spans and key-size diffusion spans are 2 KiB, matching the row geometry.

Five ARX phases and two key-size diffusions form the path in Figure 20. The first two phases write Arcane and Plasma. Arcane then controls diffusion of Plasma into Rainbow. Two further phases write Water and Vapor; Water controls diffusion of Vapor into Frost. The fifth ARX phase writes Ice A-D. These are working buffers for the row update, not separately returned material blocks.

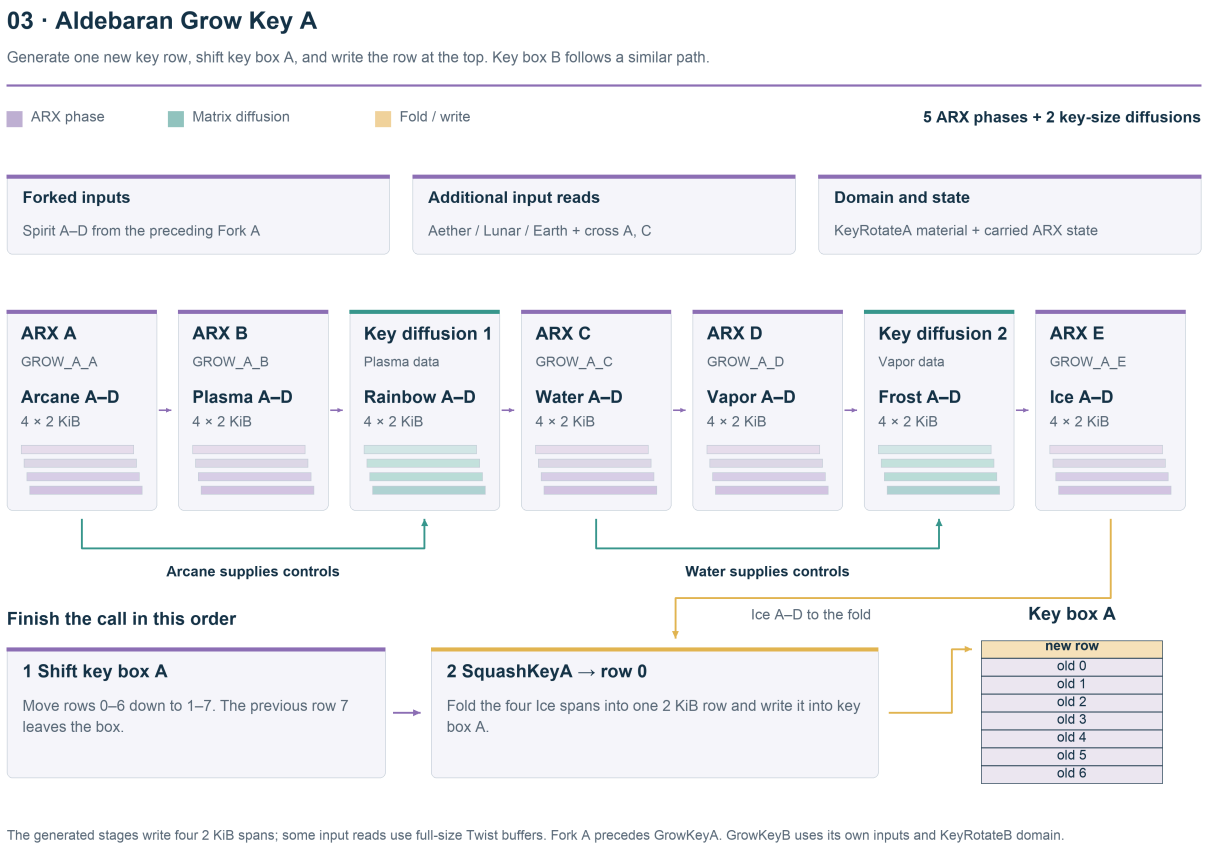


Figure 20: GrowKeyA creates a row through five ARX phases and two key-size diffusions, shifts the box, and folds Ice A-D into the new top row.

The final operations run in a fixed order: shift key box A, then fold Ice A-D with `SquashKeyA` into row 0. Rows 0-6 move down, replacing the previous row 7. The result is one new 2 KiB row alongside seven retained rows.

Key box B follows the same outer schedule: five ARX phases, two key-size diffusions, a shift, and a row write. It uses its own generated operations, KeyRotate B domain, forked material, and the caller's cross inputs B and D. The two paths are structurally similar without sharing the same row contents.

Call walkthrough: TwistBlock

A TwistBlock call accepts one 32 KiB source, four 32 KiB cross inputs, the assigned workspace, and carried ARX parameters. Its output path has **three ARX phases and one full-size diffusion**. The first phase reads row 7 of both key boxes and writes Aether A-D. The second writes Lunar A-D. Matrix diffusion uses Lunar as data and Aether as controls to produce Earth A-D; the third ARX phase writes Crystal A-D. Source and cross inputs participate throughout the generated phases, alongside earlier workspace results.

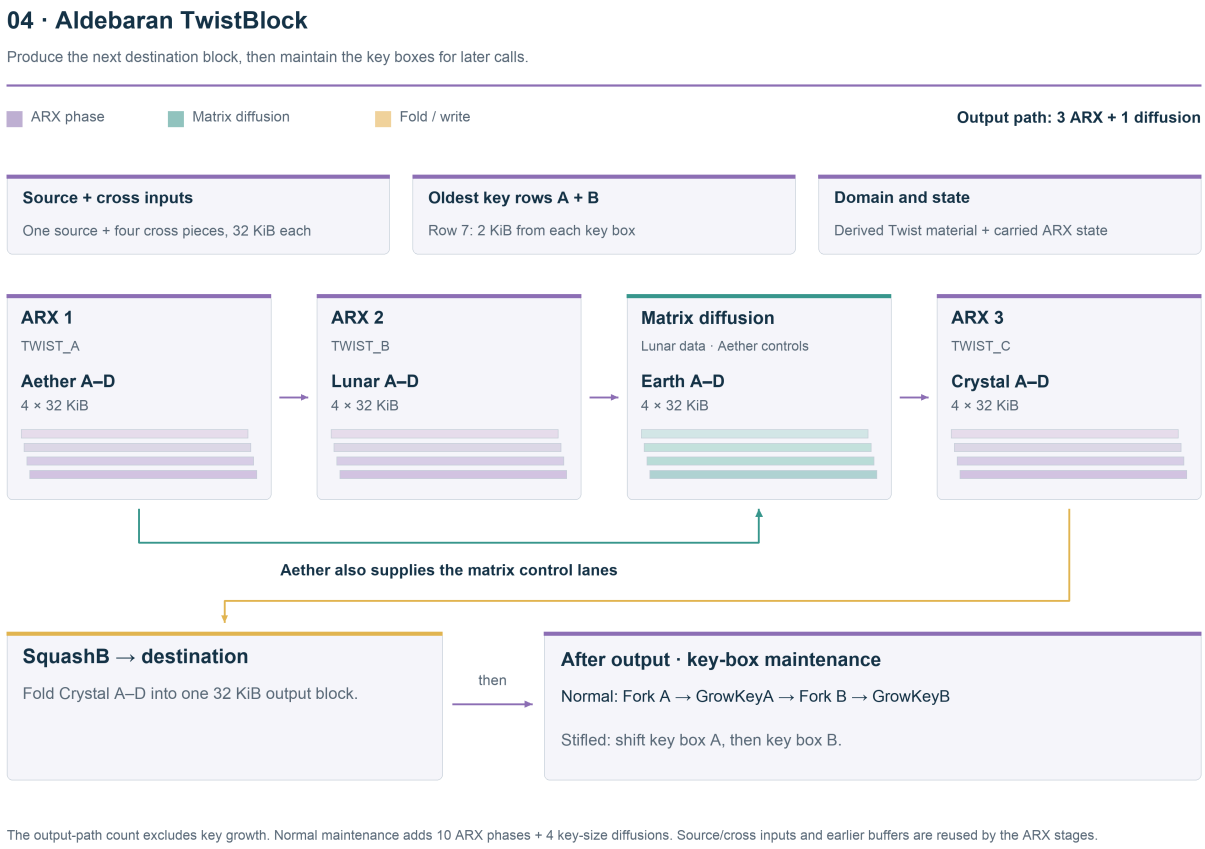


Figure 21: TwistBlock writes its destination before key maintenance. The normal branch forks and grows A, then B; the stifled branch shifts both boxes without generating replacement rows.

`SquashB` folds Crystal A-D into the destination block. Normal maintenance then runs Fork A, GrowKeyA, Fork B, and GrowKeyB. Counting those growth calls, the complete normal Twist contains **13 ARX phases, one full-size diffusion, and four key-size diffusions**. The output is already written before these updates prepare rows for later calls.

When key growth is stifled, the call shifts both boxes without inserting new rows. In the regular composer this applies to the final eight main rounds, which consume the remaining precomputed row history. This flag changes maintenance, not the three-ARX/one-diffusion output path. The caller can then select another compatible twister and workspace for the next call; the full material schedule determines which history and inputs are combined.

Security scope and fixed-plan behavior

Soccer's expanders are intended to make password recovery and output prediction computationally infeasible. Their discarded state and many-to-one folds do not establish this property. **The encryption itself is reversible** with the matching plan; the desired one-way property concerns deriving the secret from exposed outputs. An exposed complete plan permits decryption even if the original password remains unknown. Round counts and functional tests do not establish an unbreakable construction or a quantified security margin.

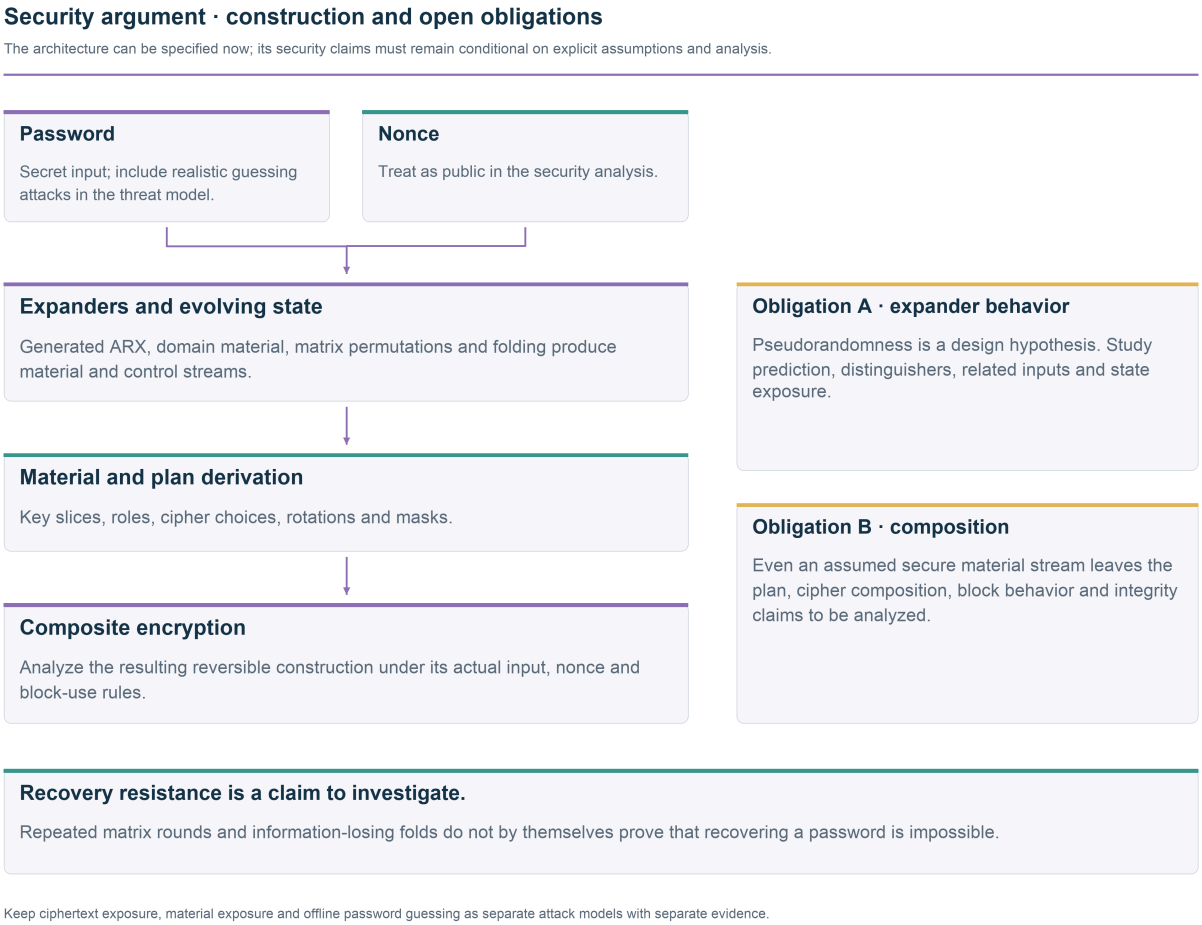


Figure 22: Two separate security obligations: analyze the expander's generated material, then analyze the resulting cipher composition under its stated input and exposure model.

Fixed-plan behavior. With the same configured plan, equal complete 1 MiB inputs produce equal ciphertexts. This equality is observable under repeated encryption queries and must be included in the construction's security model. The reproducibility needed for decryption does not itself establish confidentiality. Analysis must examine chosen inputs, related passwords or nonces, exposure of final material, and compromise of the retained plan as distinct settings.

ARX lineage and security boundary. Soccer draws on addition, rotation, and XOR, but its generated expanders and role-based composition are a separate construction from ChaCha20-Poly1305, whose encryption and authentication are specified together in RFC 8439, Section 2.8. Sharing ARX operations does not transfer that construction's security analysis. The forward and inverse transformations described here do not constitute a demonstrated authenticated-encryption scheme. The paper's security claim concerns the encryption construction under explicit assumptions; its use within an application requires its own analysis.

Proposed contribution and future work

The proposed contribution is a password-driven encryption construction in which generated expanders supply both bulk key material and the controls that select and configure a constrained composition of reversible operations. Aldebaran illustrates the single-expander machinery; the composer coordinates multiple compatible expanders, workspaces, and material lanes before binding cipher roles, masks, rotations, and key slices. This connection between material generation and cipher composition is the subject of the paper.

The design arose through implementation, experiments, and performance-guided choices in response to the requirements of the intended tool. Those choices provide a concrete engineering rationale. Establishing novelty still requires a systematic comparison with prior expander, key-derivation, and cipher constructions. Structural diversity and many rounds express design intent; they do not by themselves quantify security or show that the construction is stronger than a simpler alternative.

Specification and reproducibility. Freeze a source revision and publish deterministic test vectors for Seed, TwistBlock, complete expansion, plan construction, and encryption/decryption. Specify byte order, integer widths, overflow behavior, input restrictions, and all strength-dependent choices. Compare scalar and optimized implementations for identical results, including boundary cases and the inverse behavior of every retained primitive.

Measurements and security analysis. Report setup cost separately from steady-state encryption and decryption throughput. Record processor, compiler, build flags, strength, memory use, and workload for each result. Comparative benchmarks need equivalent input sizes and clearly stated security services. Statistical and influence-propagation experiments can identify patterns and implementation errors; independent cryptanalysis must assess output prediction, related inputs, password-guessing shortcuts, exposed state, and the complete cipher composition. The amount of expanded material is not additional password entropy. No measured security margin follows from material size or round count alone.

Revision in response to vulnerabilities. If a useful distinguisher, recovery attack, or implementation weakness is found, reproduce it and state the affected inputs, strengths, and exposure conditions. Revise or replace the relevant component, publish regression vectors, and reassess the complete composition. Revisions should carry explicit algorithm identifiers so that results and test vectors remain attributable to the construction evaluated. This is an ongoing research process, not a claim that further changes will never be necessary.

Possible communication use. A future variant could target real-time communication, but the present branch-heavy implementation makes no constant-time claim. Such a variant would need secret-independent control flow and memory accesses, as well as a separate latency evaluation. Constant time concerns dependence on secrets, not identical elapsed time on every call; these side-channel concerns also matter for local encryption (RFC 8439, Section 4). Authentication, nonce management, and protocol rules would need separate design and evaluation. The present work specifies the encryption's components and their coordination, and identifies the evidence needed to assess them.